

Experiment 06 – Advanced SQL Queries

Sahil Goyal | UID: 24BDA70148

Aim

To write SQL queries that use conditional aggregation, grouping, subqueries, and hierarchical classification of parent-child data.

Objective

- Compute the day-wise difference between two categories of sales.
- Use conditional aggregation (CASE inside an aggregate) on grouped rows.
- Understand parent-child relationships in a self-referencing table.
- Classify each node as Root, Inner, or Leaf using SQL logic.
- Order the final result as required by each question.

Question 1

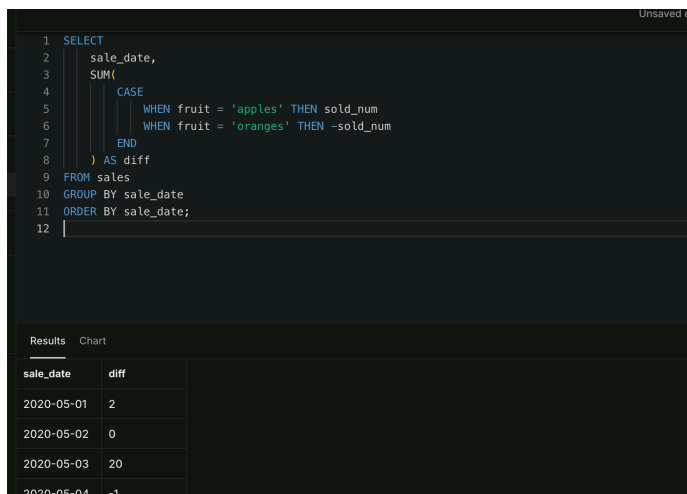
A fruit distributor logs the number of apples sold and the number of oranges sold, one row per fruit per day — so every date has exactly two rows, one for each fruit.

Build a daily comparison report showing, for each date, ApplesSold minus OrangesSold on that date. Display the sale date with the computed difference, sorted by date ascending.

Query

```
SELECT
    sale_date,
    SUM(
        CASE
            WHEN fruit = 'apples' THEN sold_num
            WHEN fruit = 'oranges' THEN -sold_num
        END
    ) AS diff
FROM sales
GROUP BY sale_date
ORDER BY sale_date;
```

Output



The screenshot shows a SQL IDE with a dark theme. The top part displays the SQL query, and the bottom part shows the results in a table. The query is the same as the one in the 'Query' section. The results table has two columns: 'sale_date' and 'diff'. The data rows show the difference for each date from 2020-05-01 to 2020-05-04.

sale_date	diff
2020-05-01	2
2020-05-02	0
2020-05-03	20
2020-05-04	-1

Question 2

A company keeps an employee reporting table where each row is one node: id is the node itself, and p_id points to its parent (NULL if it has none).

Classify every node as Root (no parent), Leaf (has a parent but no children), or Inner (has both a parent and at least one child). Display the node ID with its category, sorted by ID.

Query

```
SELECT
  id,
  CASE
    WHEN p_id IS NULL THEN 'Root'
    WHEN id IN (
      SELECT p_id
      FROM tree
      WHERE p_id IS NOT NULL
    ) THEN 'Inner'
    ELSE 'Leaf'
  END AS type
FROM tree
ORDER BY id;
```

Output

Unsaved e

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

SELECT
 id,
 CASE
 WHEN p_id IS NULL THEN 'Root'
 WHEN id IN (
 SELECT p_id
 FROM tree
 WHERE p_id IS NOT NULL
) THEN 'Inner'
 ELSE 'Leaf'
 END AS type
FROM tree
ORDER BY id;

Results Chart

id	type
1	Root
2	Inner
3	Leaf
4	Leaf
5	Leaf

Learning Outcome

- Used conditional aggregation to derive values from grouped records.
- Applied CASE expressions for conditional classification.
- Used GROUP BY and ORDER BY effectively.
- Identified parent-child relationships in hierarchical data.
- Classified hierarchical nodes as Root, Inner, or Leaf using SQL.
- Used a subquery to determine whether a node acts as a parent.